Clinical Synthetic Data Generation Harness (Realistic, Privacy-Conscious EHR Data)

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary 4

The outcome . . . 4

The problem . . . 4

The approach . . . 4

Why it is safe . . . 4

The bottom line . . . 4

Solution Overview 4

What this is . . . 5

How it works . . . 5

Benefits . . . 6

Limitations (read before relying on output) . . . 6

The deployment model: Assess → Design → Generate → Validate → Expand . . . 6

Requirements 7

R1. A confirmed target model **profile** (fields, cardinality, terminology bindings) . . . 7

R2. Agreed **reference distributions** for the pilot domains . . . 8

R3. A designated **isolated output directory** for generated data . . . 8

R4. A **reproducibility policy** (seed management and run manifests) . . . 8

R5. **Named AI/QA consumers** and their acceptance criteria . . . 9

R6. **Optional** analytics libraries for fidelity metrics . . . 9

Global Requirements — Synthetic Data & AI Test-Enablement Solutions 9

1. A Python data/engineering environment . . . 10

2. A target clinical data model / standard . . . 10

3. An isolated non-production test environment . . . 10

4. Privacy, governance & compliance stakeholders and policy . . . 10

5. AI/QA use cases and a pilot scope . . . 11

6. Reference/real-data access for fidelity (read-only, minimized) . . . 11

Implementation 11

Stage 0 — Confirm prerequisites (gate to start) . . . 11

Stage 1 — Assess (profile the scope) . . . 12

Stage 2 — Design (entities, relationships, scenarios) . . . 12

Stage 3 — Generate (the pilot dataset) . . . 13

Stage 4 — Validate (the pass/fail gates) . . . 13

Stage 5 — Expand (only after the pilot passes) . . . 14

Testing & Validation 14

T1. Schema conformance (structural validity) . . . 14

T2. Clinical plausibility (fidelity vs. reference) . . . 14
T3. Coverage (domains & scenarios the consumers need) . . . 15
T4. Reproducibility (seed + config determinism) . . . 15
T5. Privacy — no direct identifiers . . . 15
T6. Privacy — no leakage of reference records . . . 15
T7. Safe-by-default write behavior . . . 16
Release checklist (all must be true) . . . 16
Troubleshooting 16
The generator refuses my `--out-dir` . . . 16
Generation writes nothing . . . 17
Scope assessment reports BLOCK issues . . . 17
`scenario_config.py validate` fails . . . 17
Plausibility gate FAILs on a distribution . . . 17
Plausibility gate FAILs on an impossible value . . . 18
Coverage gate FAILs — a required scenario is under threshold . . . 18
Privacy gate FAILs — direct identifier detected . . . 18
Privacy gate FAILs — exact reference-record match (leakage) . . . 18
Reproducibility check fails — different manifest hash on re-run . . . 18
`pandas`/`numpy` not installed . . . 19
FHIR/OMOP conformance disagreements . . . 19
Best Practices 19
Privacy & safety . . . 19
Reproducibility . . . 19
Plausibility & fidelity . . . 20
Maintainability & extensibility . . . 20
Governance . . . 20
Deliverables Summary 20
Documentation . . . 21
The 5 Python tools (`scripts/`) . . . 21
Appendix A — PowerShell Scripts 23

Executive Summary

The outcome

AI and QA teams get **realistic clinical test data on demand** — synthetic patients, encounters, conditions, medications, labs, and vitals against a chosen target model (**FHIR**, **OMOP**, or a native schema) — that is **clinically plausible, reproducible, and privacy-conscious**, so they can build and evaluate safely **without ever touching real Protected Health Information (PHI)**.

The problem

Realistic clinical data is essential to test AI models and QA pipelines, but real EHR data carries privacy risk teams cannot responsibly take on into shared, non-production, AI test environments. Mock data is unrealistic; real data is unsafe. The gap stalls AI and QA work.

The approach

This solution stands up a **clinical synthetic data generation harness** and hands the team a self-guided build: five Python tools and step-by-step documentation that follow a disciplined, pilot-first model — **Assess** → **Design** → **Generate** → **Validate** → **Expand** — with explicit pass/fail gates for **plausibility** and **privacy**, and a documented rollback. Generation is **seeded and reproducible** (same config + seed ⇒ same dataset) and **safe by default** (writes only to an isolated output directory; no real PHI in the shared harness).

Why it is safe

- **No real PHI in the shared harness** — only aggregate reference statistics inform generation; synthetic records are artificial by construction.
- **Validated before use** — a dataset reaches a consumer only after it passes schema conformance, plausibility/coverage, reproducibility, and privacy checks (zero direct identifiers, zero exact reference-record matches).
- **Determinations stay with the experts** — the tools produce technical evidence; HIPAA/HITRUST de-identification, disclosure-risk acceptance, and IRB/privacy sign-off remain with the organization's privacy, compliance, and legal functions.

The bottom line

Realistic clinical test data, produced in minutes, reproducible for audits and bug reports, and privacy-conscious by design — delivered as a self-guided, tested build at a fraction of the cost of a bespoke engagement.

Solution Overview

What this is

The **Clinical Synthetic Data Generation Harness** is a self-guided build for a repeatable system that produces **clinically plausible, privacy-conscious synthetic Electronic Health Record (EHR) data** — patients, encounters, conditions, medications, laboratory results, and vital signs — against a chosen **target data model** (**FHIR**, **OMOP CDM**, or a native source schema). It exists so AI and QA teams get **realistic test data** without ever touching real Protected Health Information (PHI).

The harness is **seeded and reproducible**: the same configuration and random seed produce the same dataset, every time, so a defect is reproducible and a model evaluation is comparable across runs. It is **safe by default**: generation writes only to an **isolated, non-production output directory** you designate, and no real PHI is ever placed into the shared harness.

This solution does **not** make regulatory determinations. It gives the technical method and the evidence (schema conformance, plausibility metrics, privacy checks); the **de-identification / disclosure-risk determination and any IRB or privacy-office sign-off are deferred** to the organization's privacy, compliance, and legal functions.

How it works

The harness is built from five cooperating parts, each backed by one of the included Python tools:

1. **Scope & target-model assessment** — decide the target model (FHIR / OMOP / native), the

in-scope domains and fields, the AI/QA use cases, and the isolated output location. Produces a profiled scope you can review before generating anything.

1. **The generation harness** — a seeded, deterministic generator that emits synthetic patients

and their linked encounters, conditions, medications, labs, and vitals, shaped by configurable **reference distributions** and **scenario** definitions, written only to the isolated output directory.

1. **Plausibility & coverage report** — quantifies how clinically plausible and how complete the

generated dataset is (age/sex distributions, condition prevalence, lab ranges, coverage of required domains and scenarios) against the agreed reference.

1. **Schema & privacy validator** — checks that every record conforms to the target model's

structure and that no real-PHI patterns, no direct identifiers, and no exact reference-record matches have leaked in.

1. **Scenario / config helper** — scaffolds and validates the configuration and scenario files

that drive the harness, so runs are reproducible and reviewable.

Target model (FHIR / OMOP / native)

The harness is **target-model aware**. You pick one target and the tools adapt:

• **FHIR** — records emit as FHIR resource shapes (`Patient`, `Encounter`, `Condition`,

`MedicationRequest`, `Observation` for labs and vitals). Terminology bindings are referenced at the pattern level; **verify code systems and value sets against your FHIR version** and the server you will load into.

• **OMOP CDM** — records map to OMOP tables (`person`, `visit_occurrence`,

`condition_occurrence`, `drug_exposure`, `measurement`). Concept mappings are referenced at the pattern level; **verify concept IDs against your OMOP vocabulary version**.

- **Native** — records emit against the source system's own schema you supply, when neither standard applies.

Terminology, concept, and vocabulary specifics are **version-dependent** — the harness treats them as configuration you confirm against your environment, never as invented internals.

Benefits

- **Realistic test data on demand** — high-fidelity records shaped to real-world distributions, produced in minutes, not procured over months.
- **No real-PHI risk** — the shared harness never contains real PHI; privacy exposure is designed out, not managed after the fact.
- **Reproducible** — seed + config fully determine output, so evaluations and bug reports are comparable and repeatable.
- **Scenario-driven** — targeted cohorts (e.g. a sepsis pathway, a diabetes panel) can be generated on demand to exercise specific AI/QA use cases.
- **Extensible** — new domains, scenarios, and target models are added by configuration, not by rewriting the harness.

Limitations (read before relying on output)

- **Plausible, not real.** Synthetic data approximates real-world distributions; it does not reproduce every real correlation. Model performance on synthetic data **indicates** but does not **guarantee** real-world performance.
- **Fidelity is bounded by the reference.** Plausibility is only as good as the reference statistics or distributions you agree in Requirement 6. Without a reference, plausibility claims are unfounded.
- **Not a de-identification tool.** This generates artificial data; it does not de-identify real data, and it does not make HIPAA/HITRUST determinations — those are deferred to privacy/legal.
- **Vendor and version specifics deferred.** Where a step depends on a FHIR server, OMOP vocabulary, or a synthetic-data engine (e.g. Synthea or a commercial platform), consult that vendor's documentation and **verify against your version**.

The deployment model: Assess → Design → Generate → Validate → Expand

Every build in this category follows the same pilot-first model with explicit pass/fail gates:

1. **Assess** — confirm the six global requirements, pick the target model, profile the scope

and use cases, and designate the isolated output. Artifact: **Synthetic EHR – Scope – Pilot**.

1. **Design** — define entities, relationships, reference distributions, and the pilot scenario

suite. Artifact: **Synthetic EHR – Harness Design – Pilot**.

1. **Generate** — run the seeded harness to produce the pilot dataset into the isolated output.

Artifact: **Synthetic EHR – Dataset – Pilot – v1**.

1. **Validate** — run the plausibility/coverage report and the schema/privacy validator against

the pass/fail gates. Artifacts: **Synthetic EHR – Plausibility Report – Pilot** and **Synthetic EHR – Privacy Validation – Pilot**. **Do not expand until both gates pass.**

1. **Expand** — only after the pilot passes, widen domains, scenarios, and volume, re-validating

at each step. Artifact: **Synthetic EHR – Dataset – Expanded – v1**.

A documented **rollback** applies throughout: if a gate fails, discard the failing dataset from the isolated output and return to Design — never promote a dataset that failed a plausibility or privacy gate to any AI/QA consumer.

Requirements

This solution builds **on** the six category-wide requirements in `global-requirements-synthetic.md` — a supported **Python** environment (§1), a **target clinical data model** (§2), an **isolated non-production environment** (§3), **privacy/governance stakeholders and policy** (§4), **AI/QA use cases and a pilot scope** (§5), and **read-only, minimized reference access for fidelity** (§6). Confirm those first; do not duplicate them here.

The following are the **solution-specific** prerequisites for the generation harness. Each is stated as Description / Why required / How to verify / How to implement if missing.

R1. A confirmed target model profile (fields, cardinality, terminology bindings)

• **Description:** Beyond selecting FHIR / OMOP / native (global §2), a concrete **profile** of the

in-scope resources/tables: the exact fields, their data types and cardinality, required vs. optional, and the terminology/value sets or concept vocabularies each coded field binds to.

• **Why required:** The harness generates and the validator checks **against this profile**. Without

it, schema conformance is undefined and coded fields (conditions, meds, labs) cannot be made plausible.

• **How to verify:** Confirm the profile document exists and names, for each in-scope domain, the

fields, types, cardinality, and code system/vocabulary + version.

• **How to implement if missing:** Capture the profile from your FHIR implementation guide, OMOP

vocabulary release, or native schema; record the version. Store it as the **Synthetic EHR – Target Profile – Pilot** artifact.

R2. Agreed reference distributions for the pilot domains

• **Description:** For each in-scope domain, the reference statistics the synthetic data must match — age/sex mix, condition prevalence, medication-to-condition linkage, lab reference ranges and out-of-range rates, vital-sign ranges — sourced from governed real stats (global §6) or agreed published reference distributions.

• **Why required:** Plausibility is measured against these; without them the plausibility report has nothing to grade against and fidelity claims are unfounded.

• **How to verify:** Confirm a reference-distributions file exists covering every pilot domain, with its source and access terms recorded.

• **How to implement if missing:** Agree the reference approach with the privacy stakeholders (global §6), extract or cite the distributions, and store them as **Synthetic EHR – Reference Stats – Pilot**. Never copy real records into this file — only aggregate statistics.

R3. A designated isolated output directory for generated data

• **Description:** A specific, access-controlled **output path** inside the isolated environment (global §3) where — and **only** where — the harness writes generated datasets. Distinct from any real-PHI store, code repo, or shared drive.

• **Why required:** Safe-by-default generation depends on writing synthetic data to one controlled, isolated location; the tools refuse to write elsewhere by default.

• **How to verify:** Confirm the path exists, is inside the isolated environment, is access-controlled, and is empty of real PHI.

• **How to implement if missing:** Create the directory in the isolated environment, restrict access, and record it as the harness's `--out-dir`. Never point it at a production or PHI location.

R4. A reproducibility policy (seed management and run manifests)

• **Description:** A decision on how random **seeds** and **run manifests** are managed so any dataset can be regenerated: where seeds are recorded, how config versions are pinned, and how each run's inputs (seed, config hash, tool versions) are captured.

• **Why required:** Reproducibility — a core promise of the harness — requires that seed + config are captured and re-runnable; without a policy, "the same dataset" cannot be reproduced for a bug or an audit.

• **How to verify:** Confirm a run-manifest convention is defined and that seeds/config are version controlled (not real data — configs and seeds only).

• **How to implement if missing:** Adopt the run-manifest emitted by the generation tool, commit configs and seeds to version control, and record the policy in the design artifact.

R5. Named AI/QA consumers and their acceptance criteria

• **Description:** The specific downstream consumers of the synthetic data (a model-under-test, a QA suite, a pipeline) and **what "good enough" means to them** — which domains, which scenarios, and the fidelity/coverage thresholds they need.

• **Why required:** The pass/fail gates (Testing) are set from consumer needs; the scenario suite and coverage targets are meaningless without a consumer's acceptance criteria.

• **How to verify:** Confirm each consumer, the domains/scenarios they require, and their minimum fidelity/coverage thresholds are documented.

• **How to implement if missing:** Interview the AI/QA owners (global §5), record acceptance criteria, and fold them into the **Synthetic EHR – Harness Design – Pilot** artifact.

R6. Optional analytics libraries for fidelity metrics

• **Description:** `pandas` and `numpy` for the fidelity/coverage computations. The included tools are standard-library-first and degrade gracefully, but richer distribution comparisons benefit from these libraries.

• **Why required:** Not strictly required — the tools run without them — but they make plausibility and coverage reporting faster and more expressive on larger pilot datasets.

• **How to verify:** `python -c "import pandas, numpy"` returns without error in the target environment.

• **How to implement if missing:** `pip install pandas numpy` in the isolated environment. If they cannot be installed, the tools still run with reduced-precision fallbacks (they say so on stdout).

Global Requirements — Synthetic Data & AI Test-Enablement Solutions

These requirements apply to **every** solution in the **Synthetic** category (synthetic clinical data, hospital-twin simulation, and safe AI test enablement). Verify each before starting a solution's steps.

> ***Safety, vendor & regulatory boundary (applies to every Synthetic solution).*** *These solutions generate and validate **synthetic** (artificial) data and run in **isolated non-production** test environments. **Never place real Protected Health Information (PHI) into a shared synthetic harness, twin, or AI test environment** — where real data is touched at all it is read-only, minimized, and handled under the governing privacy controls. **Synthetic-data and digital-twin tools** (e.g. Synthea, and commercial platforms) and **AI/model frameworks** are integrated at the pattern/architectural level; specific vendor console internals are **cited or deferred** to that vendor's documentation, never invented. **Regulatory and legal interpretation** (HIPAA/HITRUST de-identification determinations, IRB/privacy-office sign-off, disclosure risk acceptance, and any ITAR/NERC/NRC or contract constraints) is **deferred to the organization's privacy, compliance,***

and legal functions *— the solutions provide the technical method and evidence, not legal determinations.*

1. A Python data/engineering environment

• **Description:** A supported **Python** environment (e.g. 3.10+) with the standard data toolchain (data frames, statistics, and the synthetic-data / validation libraries the solution uses).

• **Why required:** The automation and reusable assets are authored in Python (generation, de-identification, fidelity/utility metrics, scenario runners).

• **How to verify:** Confirm Python + the required packages install and run in the target environment.

• **How to implement if missing:** Provision the environment and install dependencies before starting.

2. A target clinical data model / standard

• **Description:** A defined **clinical data model or interoperability standard** to generate against — e.g. **FHIR**, **OMOP CDM**, or the source system's own schema — plus the fields/domains in scope (patients, encounters, conditions, medications, labs, vitals, notes, etc.).

• **Why required:** Realistic synthetic data and a credible hospital twin require a target schema and clinically plausible relationships; validation is measured against it.

• **How to verify:** Confirm the target model/standard and the in-scope domains with the AI/QA teams.

• **How to implement if missing:** Select and document the target model before generating.

3. An isolated non-production test environment

• **Description:** A **segregated, non-production** environment (compute + storage) for the synthetic harness, hospital twin, and AI test runs — separate from production and from real-PHI systems.

• **Why required:** Safe evaluation of AI solutions before broader adoption depends on isolation; it also bounds any residual disclosure risk.

• **How to verify:** Confirm the environment is isolated, access-controlled, and not connected to production PHI stores.

• **How to implement if missing:** Stand up / obtain the isolated environment before running.

4. Privacy, governance & compliance stakeholders and policy

• **Description:** The governing **privacy, data-governance, and compliance** policies and the named stakeholders (privacy officer, security, and — where real source data or de-identification claims are involved — IRB/legal) whose requirements the work must satisfy.

• **Why required:** Regulated healthcare data constraints, de-identification acceptance, and enterprise data governance are policy decisions the technical method must align to, not defaults.

• **How to verify:** Confirm the applicable policies, disclosure-risk threshold, and approvers.

• **How to implement if missing:** Identify the policies and stakeholders before generating or releasing test data.

5. AI/QA use cases and a pilot scope

• **Description:** The **AI use cases and QA scenarios** the synthetic data / twin must support, and a **pilot scope** to validate on before broader rollout.

• **Why required:** Fidelity, coverage, and scenario design are driven by the AI use cases; every solution follows an **Assess** → **Design** → **Generate/Build** → **Validate** → **Expand** model proven on a pilot.

• **How to verify:** Confirm the target AI use cases, evaluation goals, and a named pilot.

• **How to implement if missing:** Elicit the use cases and define the pilot before building the harness.

6. Reference/real-data access for fidelity (read-only, minimized)

• **Description:** Where synthetic **fidelity/utility** must be measured against reality, **read-only, minimized** access to reference statistics or a governed real dataset (under the privacy controls in §4) — or agreed reference distributions where real data cannot be used.

• **Why required:** Fidelity and validation assets compare synthetic output to real-world distributions and relationships; without a reference the claims are unfounded.

• **How to verify:** Confirm the reference source and its access/governance terms.

• **How to implement if missing:** Agree the reference approach (governed real stats vs. published reference distributions) before validating.

*The deployment model across every Synthetic solution is the same: **Assess** → **Design** → **Generate/Build** → **Validate** → **Expand**, with explicit pass/fail (fidelity/privacy) gates and a documented rollback. Automation and reusable assets are delivered as **Python** — never real PHI is written to a shared harness, twin, or AI test environment.*

Implementation

Build the harness following the house model — **Assess** → **Design** → **Generate** → **Validate** → **Expand**, **pilot-first**, with explicit pass/fail gates and a documented rollback. Every artifact is named with the `<Purpose> – <Scope> – <Stage>` convention and that exact name is reused in Testing, Troubleshooting, and the scripts.

All five Python tools ship in `scripts/`. Run each with `--help` first. Everything that writes is **dry-run by default**; you opt in with `--apply` (or `--write`), and generation only ever writes to the **isolated output directory** from Requirement R3.

Stage 0 — Confirm prerequisites (gate to start)

1. Confirm the six global requirements and the six solution-specific requirements (R1–R6).
2. Confirm the **isolated output directory** (R3) exists inside the isolated environment and is empty of real PHI. This path is `--out-dir` everywhere below.

> ***Gate to start:*** *do not proceed until R1 (target profile), R2 (reference stats), R3 (isolated out-dir), and R5 (consumer acceptance criteria) exist. Missing any of these means generation has no target, no reference to grade against, nowhere safe to write, or no definition of "good enough".*

Stage 1 — Assess (profile the scope)

1. Run the scope/assessment tool to profile inputs and produce the scope artifact:

```
python scripts/scope_assessment.py \
    --target-model fhir \
    --profile ./config/target-profile.json \
    --reference ./config/reference-stats.json \
    --out-dir /isolated/synthetic-ehr \
    --report ./out/scope-report.json
```

1. Review the report: it lists the target model, in-scope domains, whether the reference covers each domain, and whether the out-dir is safe (isolated, empty of PHI signals). Resolve every **BLOCK** before continuing.
1. Save the result as **Synthetic EHR – Scope – Pilot**.

> ***Assess gate:*** *the scope report must show **0 blocking issues** (target model resolved, all pilot domains have reference coverage, out-dir validated). Otherwise return to Requirements.*

Stage 2 — Design (entities, relationships, scenarios)

1. Scaffold and validate the harness configuration with the config/scenario helper:

```
python scripts/scenario_config.py init \
    --target-model fhir \
    --domains patients,encounters,conditions,medications,labs,vitals \
    --out ./config/harness-config.json
```

1. Define the pilot **scenario suite** — the targeted cohorts your named consumers (R5) require (e.g. `diabetes-panel`, `sepsis-pathway`). Add each with:

```
python scripts/scenario_config.py add-scenario \
    --config ./config/harness-config.json \
    --name diabetes-panel --size 200 \
    --require-condition E11 --require-lab HbA1c
```

1. Validate the finished config:

```
python scripts/scenario_config.py validate --config ./config/harness-config.json
```

1. Record the config, reference stats, scenario suite, and consumer acceptance criteria together as **Synthetic EHR – Harness Design – Pilot**. Commit config + seed policy (R4) to version control.

> ***Design gate:*** `validate` *must return **OK** — every scenario's required conditions/labs exist in the target profile (R1) and every consumer-required domain is covered.*

Stage 3 — Generate (the pilot dataset)

1. **Dry-run first** to preview volumes and see nothing is written:

```
python scripts/generate_harness.py \
    --config ./config/harness-config.json \
    --seed 20260706 \
    --patients 500 \
    --out-dir /isolated/synthetic-ehr \
    --dataset-name "synthetic-ehr-pilot-v1"
```

With no `--apply`, it prints the plan (record counts per domain, target model, seed, out-dir) and exits without writing.

1. **Generate for real** by adding `--apply`. The tool:

• Writes **only** under `--out-dir` (refuses any other path, refuses a path with PHI signals).

• Emits the dataset in the target model's shape plus a **run manifest** (seed, config hash, tool version, timestamp, record counts) so the run is reproducible (R4).

```
python scripts/generate_harness.py --config ./config/harness-config.json \
    --seed 20260706 --patients 500 --out-dir /isolated/synthetic-ehr \
    --dataset-name "synthetic-ehr-pilot-v1" --apply
```

1. Save the output as **Synthetic EHR – Dataset – Pilot – v1** and keep its run manifest.

> ***Reproducibility check:*** *re-running the exact command with the same* `--seed` *and config must produce an identical dataset (same manifest content hash). See Testing.*

Stage 4 — Validate (the pass/fail gates)

Run **both** validators against the pilot dataset. **Do not expand until both pass.**

1. **Plausibility & coverage** — grade the dataset against the reference (R2) and consumer criteria (R5):

```
python scripts/plausibility_report.py \
    --dataset /isolated/synthetic-ehr/synthetic-ehr-pilot-v1 \
    --reference ./config/reference-stats.json \
    --config ./config/harness-config.json \
    --report ./out/plausibility-pilot.json
```

Save as **Synthetic EHR – Plausibility Report – Pilot**.

1. **Schema & privacy** — confirm every record conforms to the target profile and that no direct identifiers, real-PHI patterns, or exact reference-record matches leaked in:

```
python scripts/schema_privacy_validator.py \
    --dataset /isolated/synthetic-ehr/synthetic-ehr-pilot-v1 \
    --profile ./config/target-profile.json \
    --reference ./config/reference-stats.json \
    --report ./out/privacy-pilot.json
```

Save as **Synthetic EHR – Privacy Validation – Pilot**.

> ***Validate gates (both must pass):*** *- **Plausibility gate** — every pilot domain within tolerance of the reference **and** every consumer-required scenario/domain covered at its threshold (R5). - **Privacy/schema gate** — **100%** schema conformance, **0** direct identifiers, **0** exact matches to any*

*reference record. If either fails: **rollback** — discard **Synthetic EHR – Dataset – Pilot – v1** from the isolated out-dir, return to Design, adjust config/reference, and regenerate. **Never** hand a dataset that failed a gate to an AI/QA consumer. **Determination is deferred:** the privacy validator produces technical evidence only. Acceptance of residual disclosure risk and any de-identification/IRB sign-off is the **privacy/compliance/legal** functions' decision (global §4), not this tool's.*

Stage 5 — Expand (only after the pilot passes)

1. With both pilot gates green and privacy sign-off recorded, widen scope **incrementally**: add domains/scenarios via `scenario_config.py`, increase `--patients`, and regenerate as **Synthetic EHR – Dataset – Expanded – v1**.

1. **Re-validate at every expansion** — rerun Stage 4 on each expanded dataset. Expansion does not inherit the pilot's pass; each new dataset must clear both gates.

1. Update the run manifests and version control on each expansion so every released dataset remains reproducible.

***Expand discipline:** one change at a time, re-validate, keep manifests. A failed expansion rolls back to the last passing dataset — the pilot dataset is never overwritten in place.*

Testing & Validation

These procedures verify the harness produces **schema-valid, clinically plausible, reproducible, privacy-conscious** data. Run them against **Synthetic EHR – Dataset – Pilot – v1** before any expansion, and against every expanded dataset thereafter. Each test states its **expected pass/fail**.

The two validators — `plausibility_report.py` and `schema_privacy_validator.py` — implement the Stage 4 gates. A dataset is releasable to an AI/QA consumer **only** when every test below passes **and** the privacy/compliance function has recorded its sign-off (deferred determination, global §4).

T1. Schema conformance (structural validity)

• **Procedure:** Run `schema_privacy_validator.py` against the dataset with the target profile (R1). Inspect the report's `schema` section.

• **Expected PASS: 100%** of records conform to the target model — every required field present, correct type/cardinality, coded fields drawn from the bound value set/vocabulary.

• **Expected FAIL:** any non-conformant record. **Action:** fix the generator/config mapping for that domain and regenerate; do not hand-edit the dataset.

T2. Clinical plausibility (fidelity vs. reference)

• **Procedure:** Run `plausibility_report.py` against the dataset and the reference stats (R2). Review each domain's distribution comparison.

• **Expected PASS:** every in-scope domain is within the agreed tolerance of the reference — age/sex mix, condition prevalence, medication-to-condition linkage, lab reference-range and out-of-range rates, vital-sign ranges. No impossible values (e.g. negative age, systolic below diastolic, a pediatric-only value in an adult cohort).

• **Expected FAIL:** any domain out of tolerance, or any impossible/implausible value. **Action:** adjust the reference distributions or generation config for that domain and regenerate.

T3. Coverage (domains & scenarios the consumers need)

• **Procedure:** In the plausibility report, review the `coverage` section against consumer acceptance criteria (R5).

• **Expected PASS:** every consumer-required **domain** is present and every required **scenario** (e.g. `diabetes-panel`, `sepsis-pathway`) is represented at or above its minimum count/threshold.

• **Expected FAIL:** a required domain absent or a scenario under its threshold. **Action:** add or resize the scenario via `scenario_config.py` and regenerate.

T4. Reproducibility (seed + config determinism)

• **Procedure:** Regenerate the dataset with the **same** `--seed` and the **same** config into a scratch location, then compare the two run manifests' content hashes (the manifest records seed, config hash, tool version, and per-domain record counts).

• **Expected PASS:** identical content hash — same seed + same config ⇒ identical dataset.

• **Expected FAIL:** differing hashes. **Action:** find the non-determinism (unseeded randomness, an unpinned config value, environment-dependent ordering) and fix it; reproducibility is a core promise (R4). Discard the scratch copy afterward.

T5. Privacy — no direct identifiers

• **Procedure:** Review the `identifiers` section of the schema/privacy validator report.

• **Expected PASS: 0** direct identifiers — no real names, SSNs, MRNs, addresses, or contact details; synthetic identifiers are clearly synthetic (e.g. prefixed, from a synthetic namespace).

• **Expected FAIL:** any field matching a direct-identifier or real-PHI pattern. **Action:** stop, investigate how it entered (a real value should never reach the harness), purge the dataset from the isolated out-dir, and correct the source before regenerating.

T6. Privacy — no leakage of reference records

• **Procedure:** Review the `leakage` section. The validator checks that no synthetic record is an **exact match** to a reference record and that no memorized real value from a reference sample appears verbatim.

- **Expected PASS: 0** exact matches to any reference record.
- **Expected FAIL:** one or more exact matches. **Action:** treat as a privacy incident for the isolated environment — purge the dataset, review how reference data was used (only aggregate stats should ever be in the reference file, R2), and regenerate.

T7. Safe-by-default write behavior

- **Procedure:** Run `generate_harness.py` **without** `--apply` (dry-run), then run it with `--out-dir` pointed at a non-isolated or PHI-signal path.
- **Expected PASS:** dry-run writes **nothing** and prints the plan; a non-isolated/PHI-signal `--out-dir` is **refused** with a clear error before any write.
- **Expected FAIL:** any write during dry-run, or any write to a refused path. **Action:** stop using the build until the guard is restored — this guard is what keeps real PHI out of the shared harness.

Release checklist (all must be true)

- T1 schema conformance = 100%
- T2 plausibility within tolerance for every domain
- T3 coverage meets every consumer's acceptance criteria
- T4 reproducible (identical manifest hash on re-run)
- T5 zero direct identifiers
- T6 zero exact reference-record matches
- T7 safe-by-default writes confirmed
- Privacy/compliance sign-off recorded (deferred determination — not this tool's call)

Troubleshooting

Common failure modes when building and running the harness, and how to resolve them. Artifact names match the Implementation and Testing guides.

The generator refuses my `--out-dir`

- **Symptom:** `generate_harness.py` exits with `REFUSED: out-dir ... matches PHI/production signal`.
- **Cause:** the path contains a signal like `prod`, `phi`, or `ehr-live` — the safe-by-default guard will not write synthetic data into anything that looks like a production/PHI location.
- **Resolution:** point `--out-dir` at the **isolated output directory** from Requirement R3. This

guard is intentional; do not disable it. Re-run `scope_assessment.py` if unsure the path is safe.

Generation writes nothing

- **Symptom:** the tool prints a plan and record counts, then "DRY-RUN: nothing written."
- **Cause:** dry-run is the default — writing is opt-in.
- **Resolution:** add `--apply`. Confirm the printed out-dir and record counts first (this is by design so you review before writing).

Scope assessment reports BLOCK issues

- **Symptom:** `scope_assessment.py` exits non-zero with `[BLOCK]` lines.
- **Cause:** a domain has no reference coverage (R2), the target profile does not define a domain (R1), or the out-dir matches a PHI signal (R3).
- **Resolution:** resolve each BLOCK — extend the reference stats, complete the target profile, or fix the out-dir — before Stage 2 (Design). The **Synthetic EHR – Scope – Pilot** artifact must show 0 blocking issues.

`scenario_config.py validate` fails

- **Symptom:** `RESULT: FAIL` with problems listed.
- **Common causes & fixes:**
- *"requires a condition but 'conditions' domain is not in scope"* — add `conditions` (or `labs`) to `--domains`, or remove the requirement from the scenario.
- *"'patients' domain missing"* — patients are the root entity; add them.
- *"scenario has non-positive size"* — set a positive `--size`.
- *"duplicate scenario name"* — rename; scenario names must be unique.

Plausibility gate FAILs on a distribution

- **Symptom:** `plausibility_report.py` shows `[FAIL]` for a domain (e.g. condition prevalence out of tolerance).
- **Cause:** the generation config or the reference distribution for that domain is off, or the pilot is too small for a stable estimate.
- **Resolution:** adjust the reference (R2) or the config for that domain and **regenerate** — never hand-edit the dataset. If the pilot is small, increase `--patients` so the estimate stabilizes. Widening `--tolerance` is a last resort and must be agreed with the consumer (R5).

Plausibility gate FAILs on an impossible value

- **Symptom:** `[FAIL]` under impossible-value checks (e.g. `diastolic>=systolic`, negative age/lab).
- **Cause:** a clinical invariant was violated by the generator or a bad config edit.
- **Resolution:** these are always failures regardless of the reference. Fix the generator/config so the invariant holds and regenerate. The bundled generator enforces `systolic > diastolic` and non-negative ages/labs — investigate any custom change that broke it.

Coverage gate FAILs — a required scenario is under threshold

- **Symptom:** `[FAIL] scenario:<name>: have N ... need M`.
- **Cause:** the scenario's required cohort is rarer than its size threshold at the current `--patients`, or the required condition/lab is not being generated.
- **Resolution:** raise `--patients`, increase the scenario weighting via `scenario_config.py`, or confirm the required condition code exists in the target profile (R1). Regenerate and re-check.

Privacy gate FAILs — direct identifier detected

- **Symptom:** `schema_privacy_validator.py` reports identifier hits (SSN/email/phone/MRN pattern) or a non-synthetic patient id.
- **Cause:** a real value reached the harness — this should never happen; only aggregate stats belong in the reference (R2).
- **Resolution: treat as an incident.** Purge the dataset from the isolated out-dir, trace how the value entered (usually a real record wrongly placed in the reference file), correct the source, then regenerate. Never scrub the output and ship it.

Privacy gate FAILs — exact reference-record match (leakage)

- **Symptom:** `leakage` hits — a synthetic record exactly matches a reference sample.
- **Cause:** the reference file contains real sample **records** (it should contain only aggregate statistics), or the harness over-fit and memorized a sample.
- **Resolution:** remove real records from the reference (keep only aggregates), reduce over-fitting, and regenerate. Treat any leakage as a privacy incident for the isolated environment.

Reproducibility check fails — different manifest hash on re-run

- **Symptom:** re-running the same `--seed` and config yields a different `content_hash` (Testing T4).
- **Cause:** non-determinism — unseeded randomness, an unpinned config value, or environment-dependent ordering.

- **Resolution:** ensure every random draw goes through the seeded generator, pin all config values, and avoid iteration over unordered structures in generation. Same seed + same config must produce an identical dataset (R4). Discard the scratch copy used for the check.

`pandas`/`numpy` not installed

- **Symptom:** a note on stdout that richer distribution comparisons are unavailable.
- **Cause:** the optional analytics libraries (R6) are absent.
- **Resolution:** `pip install pandas numpy` in the isolated environment. The tools still run without them at reduced precision — the note is informational, not a failure.

FHIR/OMOP conformance disagreements

- **Symptom:** records the validator passes are rejected by your FHIR server or OMOP tooling.
- **Cause:** version-specific terminology bindings, value sets, or concept IDs differ from the profile you configured.
- **Resolution:** the harness treats terminology/vocabulary as configuration you **verify against your version** (see Overview). Reconcile the target profile (R1) with your FHIR version / OMOP vocabulary release and regenerate; consult the vendor's documentation for version-specific internals.

Best Practices

Durable practices for keeping the synthetic clinical-data harness safe, plausible, and maintainable over time.

Privacy & safety

- **Keep the harness real-PHI-free — by design, not by discipline.** Only aggregate reference statistics (R2) ever enter the harness; never a real record. If a real value could reach the harness, the pipeline is wrong — fix the source, don't scrub the output.
- **Write only to the isolated output directory (R3).** Keep the safe-by-default write guard on; never disable it to "just get data out." A synthetic dataset outside the isolated environment is a governance problem even though it is artificial.
- **Defer determinations, produce evidence.** The tools give schema/plausibility/privacy evidence; de-identification, disclosure-risk acceptance, and IRB/privacy sign-off belong to privacy/compliance/legal (global §4). Record their sign-off alongside the dataset.
- **Treat any identifier hit or exact-match as an incident** (T5/T6) — purge, investigate the source, and only then regenerate.

Reproducibility

• **Version-control configs and seeds, never data.** The `<config, seed>` pair is the reproducible source of truth; the dataset is a derived artifact you can always regenerate.

• **Keep every run manifest.** Seed + config hash + tool version + record counts make a dataset auditable and a bug reproducible. Name datasets and manifests with the `<Purpose> – <Scope> – <Stage>` convention and keep them together.

• **Pin the target-model version.** FHIR version, OMOP vocabulary release, or native schema version are part of reproducibility — record them; a vocabulary bump can silently change conformance.

Plausibility & fidelity

• **Grade against an agreed reference, always.** Plausibility without a reference is an opinion. Re-confirm the reference stays representative of the population your consumers care about.

• **Prefer plausible over perfect.** Synthetic data indicates real-world behavior; don't over-fit the harness to a single reference sample — that risks memorizing and leaking it (T6).

• **Add clinical guardrails as invariants.** Encode impossible-value checks (age ≥ 0, systolic ≥ diastolic, meds linked to a plausible condition) as validator rules, not one-off reviews.

Maintainability & extensibility

• **Extend by configuration, not by forking the generator.** New domains, scenarios, and target models are config and reference additions — keep the harness generic.

• **Change one thing, then re-validate.** Every config/reference/scope change reruns both Stage 4 gates. Expansion never inherits the pilot's pass (T-series applies per dataset).

• **Scenario suites are living assets.** As consumer use cases evolve (R5), grow the scenario suite; retire scenarios no consumer needs so coverage stays meaningful.

• **Keep vendor/version specifics external.** Where a FHIR server, OMOP vocabulary, or synthetic-data engine (e.g. Synthea or a commercial platform) is involved, reference its documentation and verify against your version — don't hard-code assumed internals into the harness.

Governance

• **Pilot-first, gate-driven, rollback-ready.** Never skip the pilot, never promote a dataset past a failed gate, always keep the last passing dataset intact so a bad expansion can roll back.

• **Log releases.** Record which dataset (by name + manifest hash) went to which consumer, when, and under whose privacy sign-off — so a later question about a dataset has a traceable answer.

Deliverables Summary

Everything the buyer receives with the **Clinical Synthetic Data Generation Harness** — one page.

Documentation

• **Solution Overview** — what the harness is, how it works, the target-model options

(FHIR / OMOP / native), benefits, limitations, and the Assess → Design → Generate → Validate → Expand model.

• **Requirements Guide** — the six category-wide requirements plus six solution-specific prerequisites

(target profile, reference distributions, isolated output directory, reproducibility policy, named consumers, optional analytics libraries), each with how to verify and how to implement.

• **Implementation Guide** — exact, ordered, pilot-first steps through all five stages, every artifact

named, with the explicit pass/fail gates and rollback.

• **Testing & Validation** — seven procedures (schema, plausibility, coverage, reproducibility, two

privacy checks, safe-write behavior) each with expected pass/fail, plus a release checklist.

• **Troubleshooting** — common failure modes and resolutions.

• **Best Practices** — durable practices for privacy, reproducibility, fidelity, maintainability, and

governance.

• **Executive Summary** — the outcome, approach, and why it is safe, for decision-makers.

The 5 Python tools (`scripts/`)

All are Python 3.10+, standard-library-first, run with `--help`, and are **safe by default** (dry-run / opt-in for anything that writes; generation writes only to the isolated output directory; no real PHI).

1. `scope_assessment.py` — *Assessment / profiling.* Profiles the target model, in-scope domains,

reference coverage, and the isolated out-dir; reports blocking issues before generation. Produces **Synthetic EHR – Scope – Pilot**.

1. `generate_harness.py` — *Generation harness.* The seeded, reproducible generator that emits

synthetic patients, encounters, conditions, medications, labs, and vitals in the target model's shape — dry-run by default, `--apply` to write, only ever under `--out-dir`. Emits a run manifest. Produces **Synthetic EHR – Dataset – Pilot – v1**.

1. `plausibility_report.py` — *Fidelity & coverage report.* Grades the generated dataset against

the agreed reference distributions and consumer coverage criteria; implements the plausibility gate. Produces **Synthetic EHR – Plausibility Report – Pilot**.

1. `schema_privacy_validator.py` — *Schema & privacy validator.* Confirms 100% schema conformance

and checks for direct identifiers, real-PHI patterns, and exact reference-record leakage; implements the privacy/schema gate (evidence only — determination deferred). Produces **Synthetic EHR – Privacy Validation – Pilot**.

1. `scenario_config.py` — *Scope / config helper.* Scaffolds, edits, and validates the harness

configuration and scenario suite that drive reproducible runs. Produces the config inside **Synthetic EHR – Harness Design – Pilot**.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

generate_harness.py

plausibility_report.py

scenario_config.py

schema_privacy_validator.py

scope_assessment.py